

ZCash

Prepared by Kirill Sizov



ZCash pools

- Transparent (introduced at Zcash launch, 2016) — public, Bitcoin-like layer, uses t-addresses and familiar P2PKH / P2SH style transactions, good for compatibility with exchanges, custodians, and legacy tooling, but does not provide privacy.
- Sprout (introduced at launch, 2016) — the first shielded pool, based on the original JoinSplit / Zerocash-style zk-SNARK design, historically important, but now a legacy.
- Sapling (introduced October 29, 2018) — major shielded upgrade, made private transactions much faster and lighter, added better viewing keys and wallet usability, uses Groth16 on BLS12-381, still active, but not the main shielded pool.
- Orchard (introduced May 31, 2022) — current flagship shielded pool; uses Halo2, supports Unified Addresses, and was designed for better scalability/privacy ergonomics; today it is the dominant shielded pool.

Math primitives

Base vs Scalar field

- Base field (F_p): the field that the point coordinates (x, y) live in.
- Scalar field (F_q): the field that multipliers live in. When you write $[n] P$, n is an F_q element.

Pallas and Vesta

- Pallas used for all protocol cryptography (keys, commitments, nullifiers, addresses).
- Vesta used by Halo2 internally for polynomial commitments (the proof system itself).

Pallas's base field is Vesta's scalar field, and vice versa. They are "paired" curves.

Hash functions

- Sinsemilla
 - Used in Merkle tree , note commitment, etc.
 - Best for variable-length, long inputs
- Poseidon
 - Nullifier derivation
 - Best for small, fixed-size field element inputs
- BLAKE2b
 - Key derivation, outside the circuit
- Pallas hash_to_curve
 - Used to derive fixed generator points (diversified generators)

Pedersen commitment for values

- Each note has a secret value v and a random blinding factor rcv
- The Pedersen commitment is computed as: $cv = [v] \cdot V + [rcv] \cdot R$ where V and R are fixed generators on the Pallas curve
- The scheme is additively homomorphic: $cv_a + cv_b = cv_{a+b}$
- For a valid transaction with inputs v_{old} and outputs v_{new} :

$$\sum cv_i - cv_{balance} = [bsk] \cdot R$$

where $bsk = \sum rcv_i$ is the sum of all blinding factors

- The prover signs with bsk (binding signing key), and the verifier checks against bvk (binding verification key).

Sinsemilla commitment for note data

- Each note contains: diversified base point g_d , transmission key pk_d , value v , nullifier randomness ρ , and commitment randomness ψ
- These are serialized into a bitstring m
- The commitment uses the Sinsemilla hash with a blinding factor rcm :

$$cm = \text{SinsemillaCommit}_r(m) = \text{SinsemillaHash}(m) + [rcm] \cdot R$$

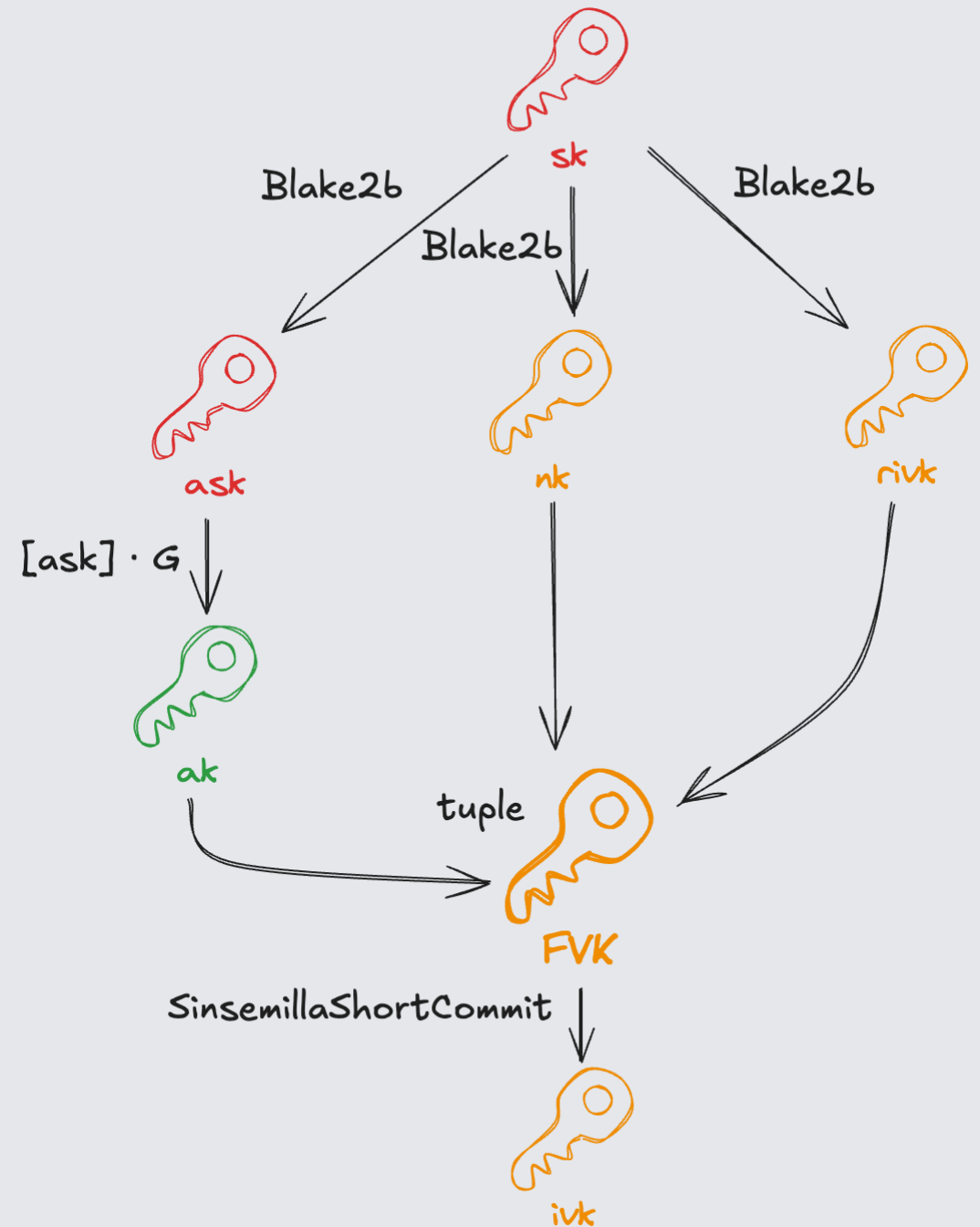
where R is a fixed generator and rcm is a random trapdoor scalar

- The on-chain value (leaf in the Merkle tree) is cmx , just the x-coordinate:

Keys

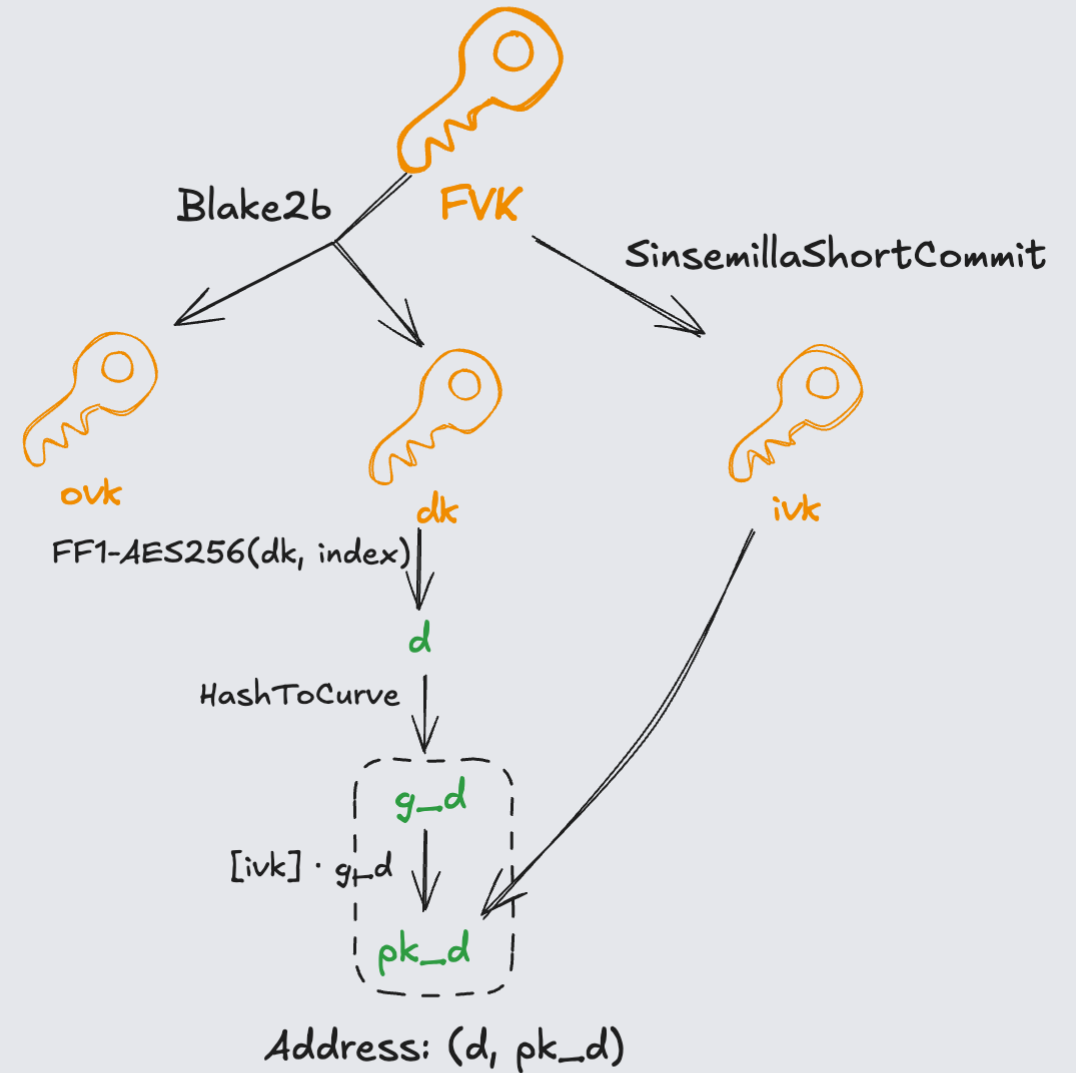
Account keys

- sk (spending key)
- ask (authorizing spending key)
- nk (nullifier key)
- fvk (full viewing key)
- rivk (randomness for incoming viewing key)
- ivk (incoming viewing key)



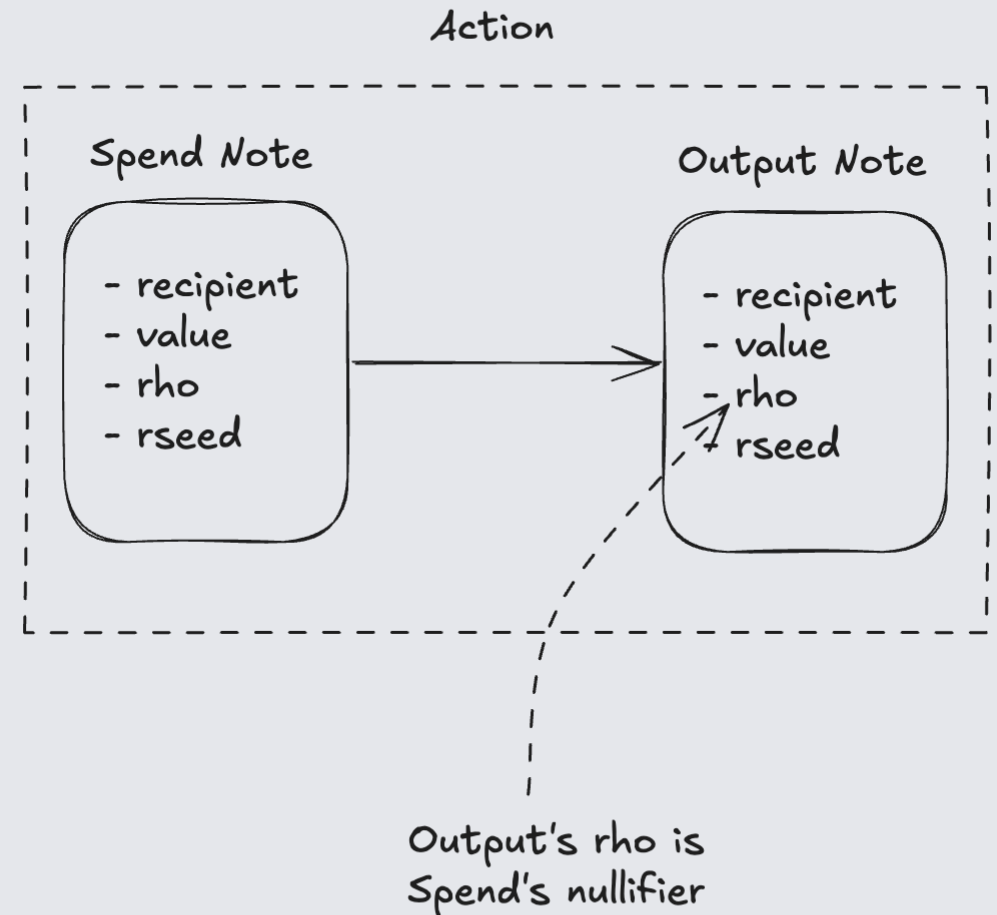
Account addresses

- dk (**d**iversifier **k**ey)
- ovk (**o**utgoing **v**iewing **k**ey)
- d (**d**iversifier)
- g_d (**g**enerator from **d**iversifier)
- pk_d (**p**ublic **k**ey, **d**iversified)



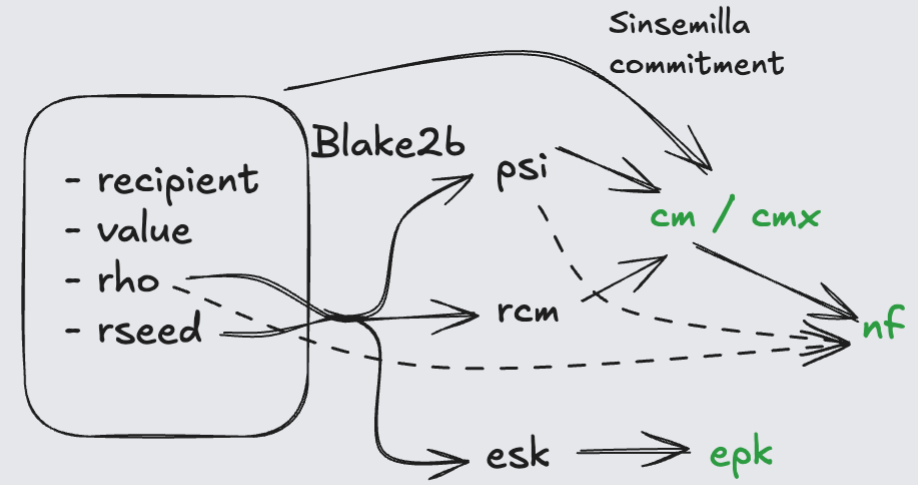
Note struct

- receiver – address of the note's owner
- value – the amount of ZEC (in zatoshis) this note is worth
- rho – prevents duplicating notes (as address, value, rseed could be duplicated)
- rseed – 32 random bytes

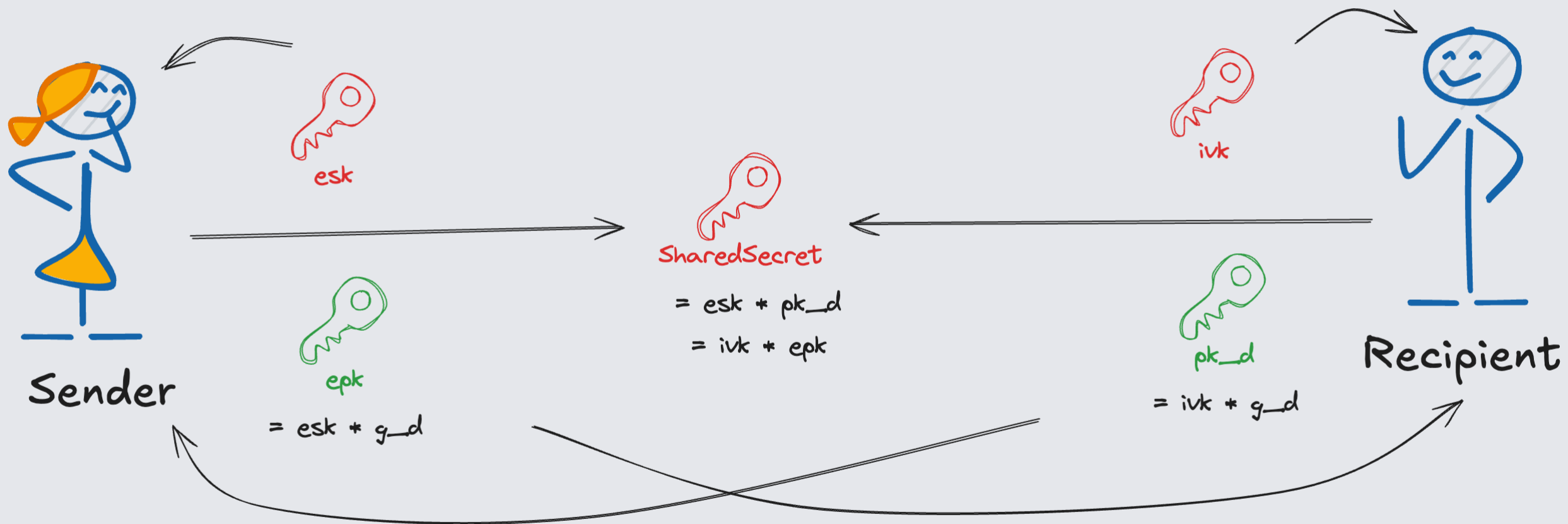


Note struct

- psi – random blinding factor
- rcm (randomness for commitment)
- esk (ephemeral secret key)
- cm (note commitment)
- cmx (x coordinate of cm)
- nf (nullifier)



ECDH



ZK circuit

Public inputs

Index	Name	What it is
0	anchor	Merkle root of the commitment tree
1-2	cv_net x,y	Value commitment (hides the transferred amount)
3	nf_old	Nullifier of the spent note
4-5	rk x,y	Randomized spend verification key
6	cmx	New note commitment (x-coordinate)
7	enable_spend	Flag: are spends allowed in this bundle?
8	enable_output	Flag: are outputs allowed?

Synthesize flow

1. Witness private inputs
2. Merkle path → `root`
3. Value commitment → public `cv_net`
4. Nullifier → public `nf_old`
5. Spend authority → public `rk`
6. Address integrity → internal `pk_d_old` check
7. Old note commitment → internal `cm_old` check
8. New note commitment → public `cmx`
9. Orchard gate → ties it all together

AddChip

a_6	a_7	a_8	q_{add}
$c = a + b$	a	b	1

Used in:

- Nullifier derivation: computes $\text{scalar} = \text{Poseidon}(nk, \rho) + \psi$

Gates:

- $a_7[0] + a_8[0] - a_6[0] = 0$

Orchard Custom Gate

Gates:

- $a_0 - a_1 - a_2 \cdot a_3 = 0$
 - $v_{\text{old}} - v_{\text{new}} = \text{magnitude} \cdot \text{sign}$
- $a_0 \cdot (a_4 - a_5) = 0$ (if $v_{\text{old}} \neq 0$
then $\text{root} = \text{anchor}$)
- $a_0 \cdot (1 - a_6) = 0$ (if $v_{\text{old}} \neq 0$
then spends enabled)
- $a_1 \cdot (1 - a_7) = 0$ (if $v_{\text{new}} \neq 0$
then outputs enabled)

a_0	a_1	a_2	a_3	a_4	a_5	a_6	a_7	q_{orch}
v_{old}	v_{new}	magnitude	sign	root	anchor	en_{spend}	$\text{en}_{\text{output}}$	1

Example: $v_{\text{old}} = 5000$, $v_{\text{new}} = 3000$, $\text{magnitude} = 2000$, $\text{sign} = 1$,
 $\text{root} = \text{anchor} = 0xab12\dots$

Orchard Custom Gate

Used in:

- Value decomposition check
- Merkle root vs anchor conditional check
- Spend/output flag enforcement

a_0	a_1	a_2	a_3	a_4	a_5	a_6	a_7	q_{orch}
v_{old}	v_{new}	magnitude	sign	root	anchor	en_{spend}	$\text{en}_{\text{output}}$	1

Example: $v_{\text{old}} = 5000$, $v_{\text{new}} = 3000$, magnitude = 2000, sign = 1, root = anchor = 0xab12...

LookupRangeCheck

Lookup argument:

- $w_i = a_9[\text{cur}] - a_9[\text{next}] \cdot 2^{10} \in T_{\text{idx}}$

Gates:

- $z_{i+1} \cdot 2^{10} + w_i = z_i$ (running sum relation)
- $z_n = 0$ (copy-constrained to constant zero)

Row	a_9 (running sum)	Derived word w_i	Lookup into T_{idx}	q_{rc}
0	$z_0 = 70000$	$w_0 = z_0 - z_1 \cdot 2^{10} = 368$	$368 \in T_{\text{idx}}$	1
1	$z_1 = 68$	$w_1 = z_1 - z_2 \cdot 2^{10} = 68$	$68 \in T_{\text{idx}}$	1
2	$z_2 = 0$			0

Row	a_9	Derived word w_i	Lookup into T_{idx}	q_{rc}
0	$z_0 = 843$	$w_0 = 843$	$843 \in T_{\text{idx}}$	1
1	$z_1 = 0$			0

T_{idx}
0
1
2
...
1022
1023

LookupRangeCheck

Used in:

- NoteCommitChip: range-check sub-pieces (b_0 to 4 bits, b_3 to 4 bits, d_2 to 8 bits, etc.)
- CommitlvkChip: range-check sub-pieces (b_0 to 4 bits, b_2 to 5 bits, d_0 to 9 bits)
- EccChip: scalar decomposition 3-bit windows
- Canonicity overflow values (a' , $b2c'$)

Row	a_9 (running sum)	Derived word w_i	Lookup into T_{idx}	q_{rc}
0	$z_0 = 70000$	$w_0 = z_0 - z_1 \cdot 2^{10} = 368$	$368 \in T_{idx}$	1
1	$z_1 = 68$	$w_1 = z_1 - z_2 \cdot 2^{10} = 68$	$68 \in T_{idx}$	1
2	$z_2 = 0$			0

Row	a_9	Derived word w_i	Lookup into T_{idx}	q_{rc}
0	$z_0 = 843$	$w_0 = 843$	$843 \in T_{idx}$	1
1	$z_1 = 0$			0

T_{idx}
0
1
2
...
1022
1023

PoseidonChip

Used in:

- Nullifier PRF: $\text{Poseidon}(nk, \rho)$

Gates:

- Full round:

$$s_i^{(\text{next})} = \text{MDS}\left((s_0 + rc_{a,0})^5, (s_1 + rc_{a,1})^5, (s_2 + rc_{a,2})^5\right)_i + rc_{b,i}$$

- Partial round:

$a_5 = (a_6 + rc_{a,0})^5$, then

$$s_i^{(\text{next})} = \text{MDS}(a_5, s_1 + rc_{a,1}, s_2 + rc_{a,2})_i + rc_{b,i}$$

Pad-and-add (absorb)

a ₅	a ₆	a ₇	a ₈	F ₂	F ₃	F ₄	F ₅	F ₆	F ₇	q _{pad}
	s _{0,init}	s _{1,init}	s _{2,init}							1
	s ₀ + nk	s ₁ + ρ	s ₂							0

Full rounds (first 4)

a ₅	a ₆	a ₇	a ₈	F ₂	F ₃	F ₄	F ₅	F ₆	F ₇	q _{full}
	s ₀ ⁽⁰⁾	s ₁ ⁽⁰⁾	s ₂ ⁽⁰⁾	rc _{a,0}	rc _{a,1}	rc _{a,2}	rc _{b,0}	rc _{b,1}	rc _{b,2}	1
	s ₀ ⁽¹⁾	s ₁ ⁽¹⁾	s ₂ ⁽¹⁾	rc _{a,0}	rc _{a,1}	rc _{a,2}	rc _{b,0}	rc _{b,1}	rc _{b,2}	1
	s ₀ ⁽²⁾	s ₁ ⁽²⁾	s ₂ ⁽²⁾	...	...	...	...	...	...	1
	s ₀ ⁽³⁾	s ₁ ⁽³⁾	s ₂ ⁽³⁾	...	...	...	...	...	...	1

Partial rounds

a ₅	a ₆	a ₇	a ₈	F _{2..4}	F _{5..7}	q _{partial}
s ₀ ⁵	s ₀ ⁽⁴⁾	s ₁ ⁽⁴⁾	s ₂ ⁽⁴⁾	rc _a	rc _b	1
s ₀ ⁵	s ₀ ⁽⁵⁾	s ₁ ⁽⁵⁾	s ₂ ⁽⁵⁾	rc _a	rc _b	1
...	...	...	...	...	...	...
s ₀ ⁵	s ₀ ⁽⁵⁹⁾	s ₁ ⁽⁵⁹⁾	s ₂ ⁽⁵⁹⁾	rc _a	rc _b	1

Last 4 full rounds + output

a ₅	a ₆	a ₇	a ₈	q _{full}
	s ₀ ⁽⁶⁰⁾	s ₁ ⁽⁶⁰⁾	s ₂ ⁽⁶⁰⁾	1
	s ₀ ⁽⁶¹⁾	s ₁ ⁽⁶¹⁾	s ₂ ⁽⁶¹⁾	1
	s ₀ ⁽⁶²⁾	s ₁ ⁽⁶²⁾	s ₂ ⁽⁶²⁾	1
	s ₀ ⁽⁶³⁾	s ₁ ⁽⁶³⁾	s ₂ ⁽⁶³⁾	1
	s _{0,out}	s _{1,out}	s _{2,out}	0

SinsemillaChip

Lookup argument:

$$(z_{\text{cur}} - z_{\text{next}} \cdot 2^{10}, a_1[\text{cur}], y_P) \in (T_{\text{idx}}, T_x, T_y)$$

Gates:

- Incomplete add step 1:

$$\lambda_1^2 - x_A - x_P - x_R = 0$$

- Incomplete add step 2:

$$\lambda_2^2 - x_R - x_A - x_{A,\text{next}} = 0$$

- y-consistency:

$$\lambda_1 \cdot (x_A - x_R) = y_A + y_P; \lambda_2 \cdot (x_A - x_{A,\text{next}}) = y_A + y_R$$

- Running sum:

$$z_{\text{next}} \cdot 2^{10} + m_i = z_{\text{cur}}$$

- Final: $z_n = 0$

Row	a ₀	a ₁	a ₂	a ₃	a ₄	Lookup (T _{idx} , T _x , T _y)
0	$x(Q)$	$x(S(m_0))$	$\lambda_{1,0}$	$\lambda_{2,0}$	$z_0 = m_{\text{packed}}$	$(m_0, x(S(m_0)), y(S(m_0)))$
1	$x(\text{Acc}_1)$	$x(S(m_1))$	$\lambda_{1,1}$	$\lambda_{2,1}$	z_1	$(m_1, x(S(m_1)), y(S(m_1)))$
2	$x(\text{Acc}_2)$	$x(S(m_2))$	$\lambda_{1,2}$	$\lambda_{2,2}$	z_2	$(m_2, x(S(m_2)), y(S(m_2)))$
3	$x(\text{Acc}_3)$				$z_3 = 0$	

T _{idx}	T _x	T _y
0	$x(S(0))$	$y(S(0))$
1	$x(S(1))$	$y(S(1))$
2	$x(S(2))$	$y(S(2))$
...	...	...
1023	$x(S(1023))$	$y(S(1023))$

SinsemillaChip

Used in:

- Commitlvk (51 chunks), old NoteCommit (109 chunks), MerkleCRH at even levels (52 chunks) ($a_0..a_4$)
- New NoteCommit (109 chunks), MerkleCRH at odd levels (52 chunks) ($a_5..a_9$)

Row	a_0	a_1	a_2	a_3	a_4	Lookup (T_{idx} , T_x , T_y)
0	$x(Q)$	$x(S(m_0))$	$\lambda_{1,0}$	$\lambda_{2,0}$	$z_0 = m_{\text{packed}}$	$(m_0, x(S(m_0)), y(S(m_0)))$
1	$x(\text{Acc}_1)$	$x(S(m_1))$	$\lambda_{1,1}$	$\lambda_{2,1}$	z_1	$(m_1, x(S(m_1)), y(S(m_1)))$
2	$x(\text{Acc}_2)$	$x(S(m_2))$	$\lambda_{1,2}$	$\lambda_{2,2}$	z_2	$(m_2, x(S(m_2)), y(S(m_2)))$
3	$x(\text{Acc}_3)$				$z_3 = 0$	

T_{idx}	T_x	T_y
0	$x(S(0))$	$y(S(0))$
1	$x(S(1))$	$y(S(1))$
2	$x(S(2))$	$y(S(2))$
...	...	...
1023	$x(S(1023))$	$y(S(1023))$

MerkleChip

Gates:

- Swap: if position bit = 1, left and right are swapped before packing
- Layer encoding: first chunk c_0 encodes the 10-bit layer index l

Used in:

- Merkle path verification: 32 levels, from leaf cmx_{old} to root

a_0	a_1	a_2	a_3	a_4	Lookup (T_{idx}, T_x, T_y)	q_s
$x(Q_{merkle})$	$x(S(c_0))$	λ_1	λ_2	z_0	$(c_0, x(S(c_0)), y(S(c_0)))$	1
$x(Acc_1)$	$x(S(c_1))$	λ_1	λ_2	z_1	$(c_1, x(S(c_1)), y(S(c_1)))$	1
...	...	...	...	...	...	...
$x(Acc_{51})$	$x(S(c_{51}))$	λ_1	λ_2	z_{51}	$(c_{51}, x(S(c_{51})), y(S(c_{51})))$	1
$x(Acc_{52}) = node_x$				$z_{52} = 0$		0

CommitlvkChip

Used in:

- Computing $ivk = \text{SinsemillaShortCommit}^{ivk}(ak || nk; rivk)$

EccChip: incomplete addition

a_0	a_1	a_2	a_3	a_4	a_5	q_{inc}
x_p	y_p	x_q	y_q	x_r	y_r	1

Used in:

Inside SinsemillaChip (every hash step is two incomplete additions) and variable-base scalar mul (double-and-add steps)

Gates:

- $(x_q - x_p) \cdot \lambda = y_q - y_p$ (λ expressed implicitly through the coordinates)
- $x_r = \lambda^2 - x_p - x_q$
- $y_r = \lambda \cdot (x_p - x_r) - y_p$

EccChip: complete addition

a_0	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	q_{add}
x_p	y_p	x_q	y_q	x_r	y_r	λ	α	β	γ	1

- $\lambda \cdot (x_q - x_p) = y_q - y_p$ (chord slope when $P \neq Q$)
- $\lambda \cdot 2y_p = 3x_p^2$ (tangent slope when $P = Q$)
- $x_r = \lambda^2 - x_p - x_q$
- $y_r = \lambda \cdot (x_p - x_r) - y_p$
- $\alpha \cdot (x_q - x_p) = 1 - \delta$ ($\delta = 1$ when $x_p = x_q$)
- $\beta \cdot (y_q + y_p) = 1 - \epsilon$ ($\epsilon = 1$ when $y_q = -y_p$)
- Identity handling: if P or Q is identity, result = the other point

EccChip: variable-base scalar mul

Gates:

- $\text{bit}_i \cdot (1 - \text{bit}_i) = 0$ (each bit is boolean)
- If $\text{bit} = 1$:
 $\text{Acc}_{\text{next}} = [2]\text{Acc} + P$
(double and add)
- If $\text{bit} = 0$: $\text{Acc}_{\text{next}} = [2]\text{Acc}$
(just double)
- Expressed as: doubling always computed via λ_1 , then P conditionally added via λ_2

Row	$\mathbf{a_0}$	$\mathbf{a_1}$	$\mathbf{a_2}$	$\mathbf{a_3}$	$\mathbf{a_4}$	$\mathbf{a_5}$	$\mathbf{a_6}$	$\mathbf{q_{mul_var}}$
0	x_{acc}	y_{acc}	x_P	y_P	λ_1	λ_2	bit_{254}	1
1	x'_{acc}	y'_{acc}	x_P	y_P	λ_1	λ_2	bit_{253}	1
...	...	...	...	...	...	...	...	...
254	x_{final}	y_{final}	x_P	y_P	λ_1	λ_2	bit_0	1

NoteCommitChip

Decomposes the 1086-bit NoteCommit message $(x(g_d) || x(pk_d) || v || \rho || \psi)$ into Sinsemilla pieces with boundary piece sub-decompositions.

